

- › Instalar módulos

- ✓ limpieza de la data

- ✓ Validaciones iniciales

Total de cards: 68

```
<div class="property-card_container property-card_highlighted"><div class="property-card_content"><a
href="/inmueble/arriendo-apartamento-bogota-sucre-1-habitaciones-1-banos/212-M6376734?src_url=%2Fapartamento-
apartastudio%2Ffarriendo%2Fbogota%2F" rel="noreferrer" target="_blank"><div class="property-card_photo"></div class="property-
card_photo-tags"><pt-tag big-type="clientes" class="hydrated" element-id="highlightTag" icon="crown" size="small" type="blue">
<div id="highlightTag"><div class="pt-tag pt-tag-blue pt-tag-small"><pt-text class="hydrated"><slot></slot></pt-text><pt-icon
class="hydrated"></pt-icon></div></div></pt-tag></div><div class="property-card_detail"><div class="property-
card_detail-top"><div class="property-card_detail-top-left"><div>Sucre | Bogotá D.C.</div></div><div class="flex shrink-0
items-center"><pt-button class="hydrated" color-style="primary-50" element-id="favorite-212-M6376734" icon="buyers/heart"
size="medium" types="tertiary"><div class="pt-button_container"><button class="pt-button pt-button--tertiary pt-button--medium
pt-button--primary-50 pt-button--primary-50--tertiary pt-button--nolabel" id="favorite-212-M6376734"><pt-icon class="pt-
button_icon hydrated"></pt-icon></button></div></pt-button></div></div><div class="flex gap-1 h-6"></div><div class="property-
card_detail-price">$1.800.000<div style="min-height: 45px; display: flex; align-items: center;"></div></div><div
class="property-card_detail-title"><h2>Apartamento en Arriendo, Sucre, Bogotá D.C.</h2></div><div class="property-
card_detail-specs"><pt-main-specs class="hydrated" element-id="specs-212-M6376734"><div class="pt-main-specs"><div class="pt-
main-specs--feature"><pt-icon class="hydrated"></pt-icon><pt-text class="hydrated">25,85 m²</pt-text></div><div class="pt-main-
specs--feature"><pt-icon class="hydrated"></pt-icon><pt-text class="hydrated">1 hab.</pt-text></div><div class="pt-main-specs--
feature"><pt-icon class="hydrated"></pt-icon><pt-text class="hydrated">1 bañ.</pt-text></div></div></pt-main-specs></div></div>
</a><div class="property-card_buttons"><pt-button class="hydrated" color-style="primary-50" element-id="openContactBtn-212-
M6376734" prefix-icon="logo-m2-variant" size="large" type="primary"><div class="pt-button_container"><button class="pt-button
pt-button--primary pt-button--large pt-button--primary-50 pt-button--primary-50 pt-button--full" id="openContactBtn-
212-M6376734"><pt-icon class="pt-button_prefix hydrated"></pt-icon><div class="pt-button_container_text"><pt-text
class="hydrated"><slot></slot></pt-text></div></button></div></pt-button><pt-button class="hydrated" color-style="whatsapp"
element-id="wspContact" icon="whatsapp" size="large" type="secondary"><div class="pt-button_container"><button class="pt-
```

```
button pt-button--secondary pt-button--large pt-button--whatsapp pt-button--whatsapp--secondary pt-button--full pt-button--
nolabel" id="wspContact"><pt-icon class="pt-button__icon hydrated"></pt-icon></button></div></pt-button><pt-button
class="hydrated" color-style="primary-50" element-id="callContact" icon="phone" size="large" type="secondary"><div class="pt-
button__container"><button class="pt-button pt-button--secondary pt-button--large pt-button--primary-50 pt-button--primary-50--
secondary pt-button--full pt-button--nolabel" id="callContact"><pt-icon class="pt-button__icon hydrated"></pt-icon></button>
</div></pt-button></div></div></div>
```

```
#href = cards[0].find_all("div", class_="property-card__content")[0].find_all("a")[0]["href"]
href = cards[0].find_all("a")[0]["href"]
href
```

```
'/inmueble/arriendo-apartamento-bogota-sucre-1-habitaciones-1-banos/212-M6376734?src_url=%2Fapartamento-apartaestudio%2Farriend
```

```
url_img = cards[0].find_all("img")[0]["src"]
url_img
```

```
'https://multimedia.metrocuadrado.com/212-M6376734/212-M6376734_188_p.jpg'
```

```
alt_img = cards[0].find_all("img")[0]["alt"]
alt_img
```

```
'Foto de Apartamento en Arriendo en Sucre, Bogotá D.C. con 1 habitaciones, 1 baños, área 25.85 m2 - 212-M6376734'
```

```
barrio = cards[0].find_all("div", class_="property-card__detail-top__left")[0].find_all("div")[0].get_text(strip=True).split("|
barrio
```

```
'Sucre '
```

```
titulo = cards[0].find_all("div", class_="property-card__detail-title")[0].find_all("h2")[0].get_text(strip=True)
titulo
```

```
'Apartamento en Arriendo, Sucre, Bogotá D.C.'
```

```
precio_str = cards[0].find_all("div", class_="property-card__detail-price")[0].get_text(strip=True)
precio_str
```

```
'$1.800.000'
```

```
atributos = cards[0].find_all("div", class_="pt-main-specs--feature")
list_atributos = [ a.get_text(strip=True) for a in atributos ]
```

```
def filtrar_atributos(lista_atributos):

    # se define un esquema inicial
    esquema_salida = {
        "area" : -1,
        "habitaciones" : -1,
        "banos" : -1,
        "parqueaderos" : 0
    }

    # aplciar filtros

    for item in lista_atributos:
        if 'm²' in item:
            esquema_salida['area'] = float(item.split()[0].replace(",","."))
        elif 'hab' in item:
            esquema_salida['habitaciones'] = int(item.split()[0])
        elif 'bañ' in item:
            esquema_salida['banos'] = float(item.split()[0])
        elif 'par' in item:
            esquema_salida['parqueaderos'] = int(item.split()[0])

    return esquema_salida
```

```
resultado_filtro = filtrar_atributos(list_atributos)
```

```
area = resultado_filtro["area"]
area
```

```
25.85
```

```
hab = resultado_filtro["habitaciones"]
hab
```

1

```
bano = resultado_filtro["banos"]
bano
```

1.0

```
parq_cant = resultado_filtro["parqueaderos"]
parq_cant
```

0

✓ Limpieza

```
precio = float(precio_str.replace("$", "").replace(".", ""))
precio
```

1800000.0

✓ Crear funciones

```
def get_href(card):
    """
    retorna el href del inmueble tomado
    """
    href = card.find_all("a")[0]["href"]

    return href

def get_url_img(card):
    """
    retorna la url de la imagen
    """
    url_img = card.find_all("img")[0]["src"]

    return url_img

def get_alt_img(card):
    """
    retorna la informacion de alt de la imagen
    """

    alt_img = card.find_all("img")[0]["alt"]

    return alt_img

def get_barrio(card):
    """
    retorna la informacion del barrio
    """

    barrio = card.find_all("div", class_="property-card__detail-top__left")[0].find_all("div")[0].get_text(strip=True).split("|")

    return barrio

def get_titulo(card):
    """
    retorna el titulo
    """
    titulo = card.find_all("div", class_="property-card__detail-title")[0].find_all("h2")[0].get_text(strip=True)

    return titulo

def get_precio_str(card):
    """
    retorna el precio en str
    """
```

```

precio_str = card.find_all("div", class_="property-card__detail-price")[0].get_text(strip=True)

return precio_str

def get_precio(precio_str):
    """
    retorna el precio en numeros
    """
    precio = int(precio_str.replace("$", "").replace(".", ""))

    return precio
def get_atributos(card):
    """
    Funcion que retorna los atributos de area,hab., banos. paq.
    """
    atributos = card.find_all("div", class_="pt-main-specs--feature")
    list_atributos = [ a.get_text(strip=True) for a in atributos ]

    return list_atributos

def get_area(resultado_filtro):
    """
    retorna el area del inmueble
    """
    area = resultado_filtro["area"]

    return area

def get_habitaciones(resultado_filtro):
    """
    retorna la cantidad de habitaciones
    """
    hab = resultado_filtro["habitaciones"]

    return hab

def get_bano(resultado_filtro):
    """
    retorna la cantidad de baños, existe la posibilidad de que sea 0.5 bano
    """
    bano = resultado_filtro["banos"]

    return bano

def get_parq_cant(resultado_filtro):
    """
    retorna la cantidad de parqueaderos
    """
    parq_cant = resultado_filtro["parqueaderos"]
    return parq_cant

```

```

def get_esquema(card):
    """
    """
    href = get_href(card)
    url_img = get_url_img(card)
    alt_img = get_alt_img(card)
    barrio = get_barrio(card)
    titulo = get_titulo(card)
    precio_str = get_precio_str(card)
    lista_atributos = get_atributos(card)
    resultado_filtro = filtrar_atributos(lista_atributos)
    area = get_area(resultado_filtro)
    hab = get_habitaciones(resultado_filtro)
    bano = get_bano(resultado_filtro)
    parq_cant = get_parq_cant(resultado_filtro)

    # limpieza

    precio = get_precio(precio_str)

    return [href, url_img, alt_img, barrio, titulo, precio_str, precio, area, hab, bano, parq_cant]

```

```

datos = []
for card in cards:

    formato = get_esquema(card)

    print(formato)
    datos.append(formato)

```

[Mostrar salida oculta](#)

Empieza a programar o a [crear código](#) con IA.

```
df = pd.DataFrame(datos)
```

```
df['id'] = df[0].str.extract(r'(M[A-Z0-9]+)')
```

Haz doble clic (o pulsa Intro) para editar

▼ Data Frame

- Creación de Id a partir de extracción desde el 'href'
- Asignación de nombres a las columnas
- Normalización de los datos de las columnas

▼ Normalización de Datos

```

# Asigno nombres a las columnas para que se asignen números automaticamente
df.columns = (['href', 'url_img', 'alt_img', 'barrio', 'servicio', 'precio_str', 'precio', 'area', 'habitaciones', 'banos', 'pa
# Normalizo datos de las columnas, en este caso uso capitalización para algunas columnas.
df['barrio'] = df['barrio'].str.title().str.strip()
df['servicio'] = df['servicio'].str.title().str.strip()
df['alt_img'] = df['alt_img'].str.title().str.strip()
display(df)

```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con df](#) [New interactive sheet](#)

```

# Para solucionar el dilema anterior, usamos la función regex extract, permitiendome extraer el código deseado
df['id'] = df['href'].str.extract(r'(M[A-Z0-9]+)')

```

```

#Creamos una nueva columna con el link del servicio
direccion = 'https://www.metrocuadrado.com'
df['link'] = direccion + df['href']

```

```

# Verificamos si se encuentran duplicados para validar que el ID realmente sea un pk
count_id = df.groupby('id').size()
count_id[count_id > 1]

```

[Mostrar salida oculta](#)

▼ Análisis Descriptivo

- Categoizamos las columnas para aplicar las metricas correctas
 - Categóricas (Texto): Agrupar y contar
 - Barrio
 - Numéricas Continuas (Decimales): Promedios y sumas
 - Precio
 - Area
 - banos

- Numéricas Discretas (Enteros): Frecuencias y modas
 - Habitaciones
 - Parqueaderos

✓ Requerimientos

- poner nombre a las columnas
- realizar descriptiva de los datos
 - validar los tipos de variables y que metricas se debe usara para su descriptiva
- normalizacion
- realizar gráficas para las columnas
- crear una columna para el id que se toma del href (Ejemplo el primero es 3072-M6017977)

Descriptiva

1. Cuales son los barrios de las ofertas y cual es el que mas se repite
2. Cual es el barrio con un promedio de arriendo alto, bajo y el medio
3. En terminos de area cuadrada cual es el barrio mas caro, bajo y medio
4. que influye mas en el costo, el area, el barrio o la cantidad de hab,baños o parqueaderos.

Haz doble clic (o pulsa Intro) para editar

```
# Validamos el estado actual de los datos
df.info()
```

[Mostrar salida oculta](#)

```
# Convertimos a numéricas las columnas que requieran este tipo de dato (precio, area, habitaciones, baños y parqueaderos)
num_columns = ['precio', 'area', 'habitaciones', 'banos', 'parqueaderos']
for col in num_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')
#
```

```
import numpy as np
```

```
# Reemplazamos los -1 por nulos reales (NaN) en las columnas correspondientes
df[['area', 'habitaciones', 'banos', 'parqueaderos']] = df[['area', 'habitaciones', 'banos', 'parqueaderos']].replace(-1, np.nan)
```

```
print('___Reporte de Calidad de Datos___')
print(f'\nTotal de Registros Finales: {len(df)}')
print('\n1. Valores Nulos Por Columna(Datos Faltantes en la Extracción):')
print(df.isnull().sum())
print('\n2. Resumen Estadístico Inicial (para detectar anomalías):')
# Mostrar solo las columnas numéricas
pd.options.display.float_format = '{:.2f}'.format
df[['precio', 'area', 'habitaciones', 'banos', 'parqueaderos']].describe()
```

[Mostrar salida oculta](#)

```
print('Máximos y Mínimos de los inmuebles\n')

# Unimos los dos filtros directamente usando pd.concat
df_extremos = pd.concat([
    df.loc[df['precio'] == df['precio'].min(), ['id', 'barrio', 'precio', 'link']],
    df.loc[df['precio'] == df['precio'].max(), ['id', 'barrio', 'precio', 'link']]
])
df_extremos

df_extremos.style.format({
    'link': lambda url: f'<a href="{url}" target="_blank">Link del servicio</a>'
})
```

[Mostrar salida oculta](#)

✓ Análisis de distribución geográfica

```
# 1. Calcular el número de barrios unicos
total_barrios = df['barrio'].unique()
print(f'Total de barrios: {len(total_barrios)}')
```

Total de barrios: 57

```
# 2. Obtener frecuencia absoluta(conteo) y relativa (porcentaje)
frec_absoluta = df['barrio'].value_counts()
frec_relativa = df['barrio'].value_counts(normalize=True)*100
print('Frecuencia Absoluta', frec_absoluta)
print('Frecuencia Relativa', frec_relativa)
```

[Mostrar salida oculta](#)

```
# Entregable 1
# Crear la tabla de frecuencias
tabla_frecuencias = pd.DataFrame({
    'Cantidad de ofertas': frec_absoluta,
    'Porcentaje de ofertas': frec_relativa
})
tabla_frecuencias
```

```
# Damos formato para que el porcentaje solo tenga 2 decimales
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con tabla_frecuencias](#) [New interactive sheet](#)

```
top5_max = tabla_frecuencias.head(5)
top5_min = tabla_frecuencias.tail(5)
print(" Top 5 Barrios con más oferta")
display(top5_max)
print(" Top 5 Barrios con menos oferta")
display(top5_min)
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con top5_max](#) [New interactive sheet](#) [Generar código con top5_min](#) [New interactive sheet](#)

✓ Analisis de tendencia central de precios

```
# 1. Agrupar por barrio y calcular las métricas estadísticas
resumen_barrios = df.groupby('barrio')['precio'].agg(['mean', 'median', 'min', 'max'])
resumen_barrios.columns = ['Media', 'Mediana', 'Mínimo', 'Máximo']
resumen_barrios
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con resumen_barrios](#) [New interactive sheet](#)

```
# 2. Ordenar la tabla usando la Mediana como métrica principal (de mayor a menor)
resumen_barrios = resumen_barrios.sort_values(by='Mediana', ascending=False)
resumen_barrios
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con resumen_barrios](#) [New interactive sheet](#)

```
# Damos formato para evitar la notación científica y leer en millones
pd.options.display.float_format = '{:,.2f}'.format
```

```
# 3. Clasificar los barrios en segmentos creando una nueva columna
# Usamos cuantiles para dividir la tabla en 3 partes matemáticas iguales
resumen_barrios['Segmento'] = pd.qcut(resumen_barrios['Mediana'], q=3, labels=['Segmento bajo', 'Segmento medio', 'Segmento alt
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con resumen_barrios](#) [New interactive sheet](#)

```
# 4. Mostrar el Entregable
print("--- ANÁLISIS DE PRECIOS Y SEGMENTACIÓN POR BARRIO ---")
display(resumen_barrios)
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con resumen_barrios](#) [New interactive sheet](#)

✓ **Análisis de valorización (\$/m2)**

```
# 1. Crea una columna para valorizar el m2 sobre el precio
df['valor_m2'] = df['precio']/df['area']
df
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con df](#) [New interactive sheet](#)

```
# 2. Agrupar por barrio y calcular la mediana
ranking_m2 = df.groupby('barrio')['valor_m2'].agg('median').reset_index()
ranking_m2.columns = ['barrio', 'mediana_valor_m2']
ranking_m2 = ranking_m2.sort_values(by='mediana_valor_m2', ascending=False)
ranking_m2.head(10)
```

[Mostrar salida oculta](#)

Pasos siguientes: [Generar código con ranking_m2](#) [New interactive sheet](#)

```
# Mostrar informe por ranking de barrios con el valor del metro cuadrado más elevado
print('RANKING DE BARRIOS POR VALOR M2\n(Mas caros a los más económicos)')
display(ranking_m2.head(10).style.format({'mediana_valor_m2': '{:,.2f}'))
```

[Mostrar salida oculta](#)

✓ **Análisis de determinantes del precio**

- Calcular matriz de correlación entre precio, area, habitaciones, baños y parqueaderos.
- Evaluar fuerza de relación

```
# Seleccionamos las columnas numericas
columnas_num = ['precio', 'area', 'habitaciones', 'banos', 'parqueaderos']
columnas_num
```

[Mostrar salida oculta](#)

```
# Calculamos la matriz de correlación de pearson
matriz_corr = df[columnas_num].corr()
print('---MATRIZ DE CORRELACIÓN DE PEARSON---')
# Mostramos la matriz con un mapa de calor (colores) para facilitar la lectura
display(matriz_corr.style.background_gradient(cmap='coolwarm', axis=None).format('{:,.2f}'))
```

---MATRIZ DE CORRELACIÓN DE PEARSON---

	precio	area	habitaciones	banos	parqueaderos
precio	1.00	0.84	nan	0.68	0.70
area	0.84	1.00	nan	0.75	0.71
habitaciones	nan	nan	nan	nan	nan
banos	0.68	0.75	nan	1.00	0.62
parqueaderos	0.70	0.71	nan	0.62	1.00

Análisis adicional

- Precio promedio por número de baños
- Precio promedio por número de habitaciones

```
# Precio promedio por número de baños
print("---PRECIO PROMEDIO POR NÚMERO DE BAÑOS---")
precio_por_banos = df.groupby('banos')['precio'].mean().reset_index()
display(precio_por_banos.style.format({'banos': '{:.0f}', 'precio': '${:,.2f}'}))
```

[Mostrar salida oculta](#)

```
# Precio promedio por número de habitaciones
print('---PRECIO PROMEDIO POR NÚMERO DE HABITACIONES---')
precio_por_habitacion = df.groupby('habitaciones')['precio'].mean().reset_index()
display(precio_por_habitacion.style.format({'precio': '${:,.2f}'}))
```

[Mostrar salida oculta](#)

```
habitaciones = df[['id', 'habitaciones']].max()
habitaciones
```

[Mostrar salida oculta](#)

```
# 3. Análisis Controlado: Comparar inmuebles idénticos (1 hab, 1 baño, 1 parqueadero) en distintos barrios
print("\n--- 3. ANÁLISIS CONTROLADO POR BARRIO (Inmuebles idénticos: 1 Hab, 1 Baño, 1 Parq) ---")
# Filtramos el dataset para que las características físicas sean exactamente iguales
inmuebles_similares = df[(df['habitaciones'] == 1) & (df['banos'] == 1) & (df['parqueaderos'] == 1)]
display(inmuebles_similares[['id', 'barrio', 'precio', 'area', 'servicio']])
# Agrupamos por barrio calculando la mediana para ver el "efecto ubicación"
control_barrio = inmuebles_similares.groupby('barrio')['precio'].median().sort_values(ascending=False).head(5).reset_index()
display(control_barrio.style.format({'precio': '${:,.2f}'}))
```

[Mostrar salida oculta](#)

Empieza a programar o a [crear código](#) con IA.

▾ Análisis de impacto de características

- Anlaizaar variación de precios según:

- Número de baños
 - Número de habitaciones
- Calcular diferencias promedio entre categorías

```
# Impacto del precio según el número de baños
print("---IMPACTO SEGÚN NÚMERO DE BAÑOS---")
# 1. Agrupar por baños calculando el promedio de precio y cuantos inmuebles hay
impacto_banos = df.groupby('banos')['precio'].agg(['mean', 'count']).reset_index()
impacto_banos.columns = ['cant_banos', 'precio_promedio', 'total_inmuebles']
# 3. Calcular la diferencia de precio (El "salto" de valor frente a tener un baño menos)
impacto_banos['variacion_vs_anterior'] = impacto_banos['precio_promedio'].diff()
# 4. Mostrar la tabla comparativa con formato
display(impacto_banos.style.format({
    'cant_banos': '{:.0f}',
    'precio_promedio': '${:,.2f}',
    'variacion_vs_anterior': '${:,.2f}'
})))
```

---IMPACTO SEGÚN NÚMERO DE BAÑOS---

	cant_banos	precio_promedio	total_inmuebles	variacion_vs_anterior
0	1	\$2,115,869.77	52	\$nan
1	2	\$4,008,000.00	15	\$1,892,130.23
2	3	\$5,200,000.00	1	\$1,192,000.00

```
# Impacto del precio según el número de habitaciones
print("---IMPACTO SEGÚN NÚMERO DE HABITACIONES---")
# 1. Agrupar por habitaciones calculando el promedio de precio y cuantos inmuebles hay
impacto_habit = df.groupby('habitaciones')['precio'].agg(['mean', 'count']).reset_index()
impacto_habit.columns = ['cant_habit', 'precio_promedio', 'total_inmuebles']
# 3. Calcular la diferencia de precio (El "salto" de valor frente a tener un baño menos)
impacto_habit['variacion_vs_anterior'] = impacto_habit['precio_promedio'].diff()
# 4. Mostrar la tabla comparativa con formato
display(impacto_habit.style.format({
    'cant_habit': '{:.0f}',
    'precio_promedio': '${:,.2f}',
    'variacion_vs_anterior': '${:,.2f}'
})))
```

[Mostrar salida oculta](#)

```
# Impacto del precio según el número de parqueaderos
print("---IMPACTO SEGÚN NÚMERO DE PARQUEADEROS---")
# 1. Agrupar por parqueaderos calculando el promedio de precio y cuantos inmuebles hay
impacto_parq = df.groupby('parqueaderos')['precio'].agg(['mean', 'count']).reset_index()
impacto_parq.columns = ['cant_parq', 'precio_promedio', 'total_inmuebles']
# 3. Calcular la diferencia de precio (El "salto" de valor frente a tener un baño menos)
impacto_parq['variacion_vs_anterior'] = impacto_parq['precio_promedio'].diff()
# 4. Mostrar la tabla comparativa con formato
display(impacto_parq.style.format({
    'cant_parq': '{:.0f}',
    'precio_promedio': '${:,.2f}',
    'variacion_vs_anterior': '${:,.2f}'
})))
```

[Mostrar salida oculta](#)

```
# Impacto del precio según el área
print("---IMPACTO SEGÚN EL ÁREA---")
# 1. Crear las secciones: Agrupamos la variable continua 'area' en 5 rangos o cubetas
df['rango_area'] = pd.qcut(df['area'], q=5)
impacto_area = df.groupby('rango_area', observed=False)['precio'].agg(['mean', 'count']).reset_index()
# Renombramos las columnas
impacto_area.columns = ['rango_area', 'precio_promedio', 'total_inmuebles']
impacto_area['variacion_vs_anterior'] = impacto_area['precio_promedio'].diff()
# Ajustamos el formato de la tabla
display(impacto_area.style.format({'precio_promedio': '${:,.2f}', 'variacion_vs_anterior': '{:,.2f}'})))
```

[Mostrar salida oculta](#)

Empieza a programar o a [crear código](#) con IA.